

AWS Cloud Engineer Internship Project (Beginner Level)

Serverless Contact Form Application Using Amazon S3, API Gateway and AWS Lambda

1. Project Overview

The objective of this project was to build and deploy a small cloud-hosted application using AWS services.

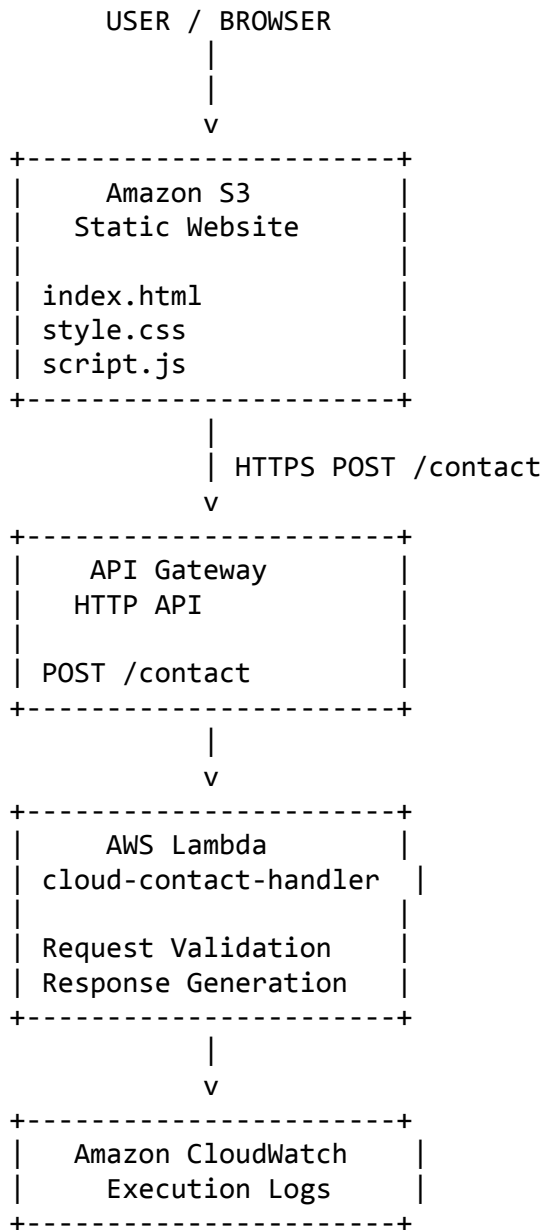
The application is a static contact/feedback website hosted on Amazon S3. Users can submit their name, email address, and message through a contact form. The submitted data is sent to an Amazon API Gateway endpoint, which invokes an AWS Lambda function for server-side validation and processing.

The project demonstrates:

- Static website hosting on AWS
 - Serverless backend architecture
 - API creation using API Gateway
 - AWS Lambda functions
 - Client-side and server-side validation
 - Cross-Origin Resource Sharing (CORS)
 - CloudWatch logging
 - Basic AWS security and cost awareness
-

2. Project Architecture

The application follows this architecture:



Request Flow

1. A user opens the static website hosted on Amazon S3.
 2. The user fills out the contact form.
 3. JavaScript validates the input on the client side.
 4. The browser sends a POST request to API Gateway.
 5. API Gateway invokes the Lambda function.
 6. Lambda validates the request again on the server side.
 7. Lambda returns a success or failure response.
 8. API Gateway returns the response to the browser.
 9. Lambda execution information is available through CloudWatch Logs.
-

3. AWS Services Used

AWS Service	Purpose
Amazon S3	Hosts the static frontend
Amazon API Gateway	Provides the HTTP API endpoint
AWS Lambda	Processes and validates form submissions
Amazon CloudWatch	Stores Lambda execution logs
AWS IAM	Provides Lambda execution permissions

4. Frontend Implementation

The frontend consists of three files:

index.html
style.css
[script.js](#)

index.html

The HTML file provides the structure of the website, including:

- Navigation/header
- Hero section
- Contact form
- Name field
- Email field
- Message field
- Submit button
- Response message
- AWS/serverless architecture section
- Footer

style.css

The CSS file provides:

- Responsive design
- Form styling
- Buttons and cards
- Validation states
- Responsive mobile layout
- AWS-inspired visual design

script.js

The JavaScript file handles:

- Form validation
- Character counting
- Error messages
- API communication
- Loading state
- Success/failure responses

The frontend communicates with API Gateway using:

```
const API_URL =  
"https://qyvjdjginwh.execute-api.us-east-1.amazonaws.com/contact";
```

The request is sent using an HTTP POST request with JSON data.

Example request:

```
{  
  "name": "Amarjit",  
  "email": "test@example.com",  
  "message": "Hello from CloudContact"  
}
```

5. Amazon S3 Configuration

An S3 bucket was created for hosting the static website.

The following files were uploaded to the root of the bucket:

```
index.html  
style.css  
script.js
```

Static website hosting was enabled with:

Index document: index.html

The bucket was configured to allow public read access to the website objects because the assignment requires the website to be publicly accessible.

A bucket policy was configured with the following principle:

Allow public s3:GetObject access

This allows visitors to retrieve the website files while not granting them permission to upload, modify, or delete objects.

Security consideration

No passwords, API keys, credentials, or sensitive information are stored in the public S3 bucket.

For a production application, Amazon CloudFront with Origin Access Control would be preferable to direct public S3 website access.

6. API Gateway Configuration

An HTTP API named:

`cloud-contact-api`

was created using Amazon API Gateway.

API Details

API Type: HTTP

Region: `us-east-1`

API ID: `qyvjdginwh`

Route

POST `/contact`

Integration

The route is connected to:

AWS Lambda

`cloud-contact-handler`

The final API endpoint used by the frontend is:

`https://qyvjdginwh.execute-api.us-east-1.amazonaws.com/contact`

The API was successfully tested independently using a POST request.

7. AWS Lambda Implementation

A Lambda function named:

`cloud-contact-handler`

was created.

The function is responsible for:

1. Receiving the API request.
2. Reading the request body.
3. Parsing the JSON data.
4. Validating the submitted fields.
5. Returning a success response for valid requests.
6. Returning an appropriate error response for invalid requests.
7. Handling unexpected errors.

Validation Rules

Field	Validation
Name	Required, minimum 2 characters, maximum 50
Email	Required and checked against an email format
Message	Required, minimum 10 characters, maximum 500

The backend validation is important because client-side validation alone cannot be trusted.

8. Lambda Response

For a valid request, Lambda returns an HTTP 200 response similar to:

```
{  
  "message": "Your message has been submitted successfully!"  
}
```

For invalid requests, the function returns an appropriate HTTP 400 response.

For example, if the request body is missing:

```
{  
  "message": "Request body is required."  
}
```

This demonstrates basic backend request validation and error handling.

9. CORS Configuration

Initially, the API used:

```
Access-Control-Allow-Origin: *
```

This allowed the frontend to communicate with API Gateway during development.

After the S3 website was successfully deployed, CORS was restricted to the actual S3 website origin.

The API allows:

Methods:

POST

OPTIONS

and:

Allowed Header:

Content-Type

Credentials were not enabled.

This allows the browser-hosted frontend to communicate with the API while reducing unnecessary cross-origin access.

10. IAM Configuration

The Lambda function was assigned an execution role with basic Lambda execution permissions.

The role allows Lambda to perform its basic execution tasks and write execution information to CloudWatch Logs.

The project follows the principle of giving services only the permissions required for their function.

No unnecessary permissions such as administrator access were required for the application runtime.

11. CloudWatch Monitoring

AWS Lambda execution logs are automatically available through Amazon CloudWatch Logs.

The Lambda log group follows the naming convention:

/aws/lambda/cloud-contact-handler

12. Testing

The application was tested at multiple levels.

Test 1 — Lambda

Lambda was tested directly using an event containing:

```
{
  "body":
  "{\\"name\\":\\"Amarjit\\",\\"email\\":\\"test@example.com\\",\\"message\\":\\"Hello from AWS Lambda\\"}"
}
```

The Lambda returned a successful response.

Test 2 — API Gateway

API Gateway was tested using:

```
{
  "name": "Amarjit",
  "email": "test@example.com",
  "message": "Hello from API Gateway"
}
```

The API successfully invoked Lambda and returned:

Your message has been submitted successfully!

Test 3 — Local Frontend

The website was run through Live Server and tested from a web browser.

CORS configuration was added to API Gateway so that the browser could successfully communicate with the API.

Test 4 — S3 Hosted Website

Finally, the website was opened directly using the S3 static website endpoint.

A test contact form submission successfully travelled through:

S3 → API Gateway → Lambda → API Gateway → Browser

and displayed the success message.

Therefore, the complete application workflow was successfully verified.

13. Security Considerations

Several basic security practices were applied.

Input Validation

Input is validated both:

- On the frontend
- On the Lambda backend

This prevents invalid requests from being blindly processed.

CORS

CORS was configured to allow the deployed frontend to communicate with the API.

S3 Permissions

The S3 bucket allows public read access only for website objects.

Users do not receive permissions to modify or delete the bucket contents.

No Sensitive Data

The static website contains no:

- AWS credentials
- API keys
- Passwords
- Private credentials
- Sensitive configuration

Least Privilege

The Lambda execution role uses basic execution permissions rather than unnecessary administrator permissions.

14. Cost Considerations

The application uses a serverless architecture and does not require an always-running EC2 server.

The main AWS components are:

Amazon S3
API Gateway
AWS Lambda
CloudWatch

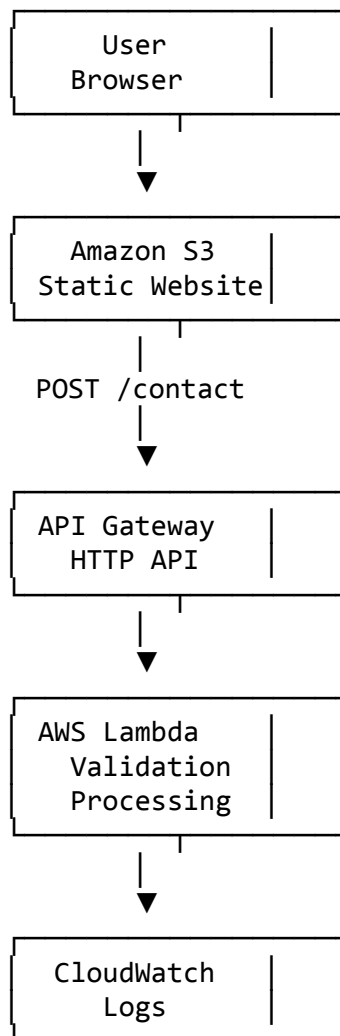
For a small internship project with low traffic, resource usage should remain low. AWS pricing and free-tier eligibility can change, so actual costs should be monitored through the AWS Billing/Cost Management console.

To avoid unnecessary charges:

- Do not create unused EC2 instances.
 - Do not leave unnecessary resources running.
 - Avoid storing large files in S3.
 - Monitor Lambda and API usage.
 - Delete the resources after the internship assignment if they are no longer required.
-

16. Final Architecture

The completed application can be summarized as:



17. Learning Outcomes

Through this project, the following AWS and cloud concepts were demonstrated:

- Static website deployment
 - Amazon S3 bucket configuration
 - S3 static website hosting
 - Bucket policies
 - HTTP APIs
 - API Gateway routes
 - AWS Lambda functions
 - Serverless architecture
 - IAM execution roles
 - CORS configuration
 - Client-side validation
 - Server-side validation
 - CloudWatch logging
 - API testing
 - Basic cloud security
 - Cost-conscious cloud deployment
-

18. Conclusion

The project successfully implements a small serverless web application using AWS.

The static frontend is hosted on Amazon S3, while API Gateway provides the HTTP interface for the contact form. AWS Lambda performs backend validation and processing, and CloudWatch provides basic execution monitoring.

The final architecture avoids the need for traditional always-running servers and demonstrates how multiple AWS managed services can be combined to build a lightweight, scalable cloud application.

The application was successfully tested from the deployed S3 website, confirming that the complete frontend-to-backend workflow is operational.

19. Screenshots

Final Website:

 CloudContact

AWS Serverless

POWERED BY AWS

Let's build something great together.

Have a question, suggestion, or project idea? Send us a message and we'll get back to you.

**Serverless**
Powered by AWS Lambda

**Secure**
Built with cloud best practices

**Cloud Hosted**
Deployed on Amazon S3

 **Send a message**
We'll respond as soon as possible.

Your name
Amarjit Bhuj

Email address
hello@gmail.com

Message
Hi, How are you?
18 / 500

Send message →

 CloudContact

AWS Serverless

POWERED BY AWS

Let's build something great together.

Have a question, suggestion, or project idea? Send us a message and we'll get back to you.

**Serverless**
Powered by AWS Lambda

**Secure**
Built with cloud best practices

**Cloud Hosted**
Deployed on Amazon S3

 **Send a message**
We'll respond as soon as possible.

Your name
John Doe

Email address
john@example.com

Message
Tell us a little about your idea...
0 / 500

Send message →

Your message has been submitted successfully!

S3 Bucket:

cloudcontact-amarjit-2026 Info

[Objects](#) | [Metadata](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [File systems](#) | [Access Points](#)

Objects (3)

Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	index.html	html	September 5, 2026, 15:37:48 (UTC+05:30)	10.0 KB	Standard
☐	script.js	js	September 5, 2026, 15:37:49 (UTC+05:30)	7.3 KB	Standard
☐	style.css	css	September 5, 2026, 15:37:50 (UTC+05:30)	11.7 KB	Standard

Lambda and API Gateway:

Lambda > Functions > cloud-contact-handler

Diagram | Template

cloud-contact-handler
 Layers (0)

API Gateway

+ Add trigger

+ Add destination

Function ARN
 `arn:aws:lambda:us-east-1:743116070426:function:cloud-contact-handler`

Description
-

Last modified
[1 day ago](#)

[Code](#) | [Test](#) | [Monitor](#) | [Configuration](#) | [Aliases](#) | [Versions](#)

Configuration

General configuration

[Triggers](#)

Permissions

Destinations

Function URL

Triggers (1) Info

< 1 >

☐	Trigger
☐	API Gateway: cloud-contact-api <code>arn:aws:execute-api:us-east-1:743116070426:qyvjdjginwh/*/*/contact</code> API endpoint: https://qyvjdjginwh.execute-api.us-east-1.amazonaws.com/contact ▶ Details

